

Spring Transactions Internals

Deep Dive Tutorial — Chapter 07

Java Backend Series

A. Transactions & ACID	Bank transfer, atomicity, consistency
B. @Transactional Basics	How the annotation wires up
C. AOP Proxy & Interceptor	TransactionInterceptor.invoke() walkthrough
D. Propagation	REQUIRED, REQUIRES_NEW, NESTED
E. Isolation Levels	Dirty/phantom/non-repeatable read
F. Rollback Rules	rollbackFor, noRollbackFor
G. Self-Invocation Trap	Why same-class calls bypass the proxy
H. Transaction Synchronisation	ThreadLocal & TransactionSynchronizationManager
I. Programmatic Transactions	TransactionTemplate
J. Read-Only Transactions	FlushMode.MANUAL, connection.setReadOnly
K. JOIN FETCH + Pagination Trap	HHH90003004 warning & fixes

A What a Transaction Is

ACID properties with a bank-transfer example

1. Plain English Explanation

A **transaction** is a group of database operations that must all succeed together or all be undone together. Without transactions, a crash halfway through a bank transfer could debit one account without crediting the other — money vanishes. Transactions give you a safe unit of work with four guarantees collectively called **ACID**.

- **Atomicity** — all operations happen or none do.
- **Consistency** — the DB moves from one valid state to another.
- **Isolation** — concurrent transactions do not interfere.
- **Durability** — once committed, data survives crashes.

2. Real-World Analogy

Think of a bank transfer as a bank teller writing two slips — one debit, one credit — and putting them in an envelope. The envelope is sealed and posted only if both slips are filled correctly. If the pen runs out mid-way, both slips are shredded and the account balances stay unchanged.

3. Minimal Working Code Example

```
// Pseudocode – illustrates atomicity
Connection conn = dataSource.getConnection();
conn.setAutoCommit(false);          // begin transaction
try {
    debit(conn, "Alice", 500);        // step 1
    credit(conn, "Bob", 500);         // step 2
    conn.commit();                    // both succeed -> persist
} catch (Exception e) {
    conn.rollback();                  // any failure -> undo ALL
} finally {
    conn.setAutoCommit(true);
    conn.close();
}
```

4. ACID Diagram

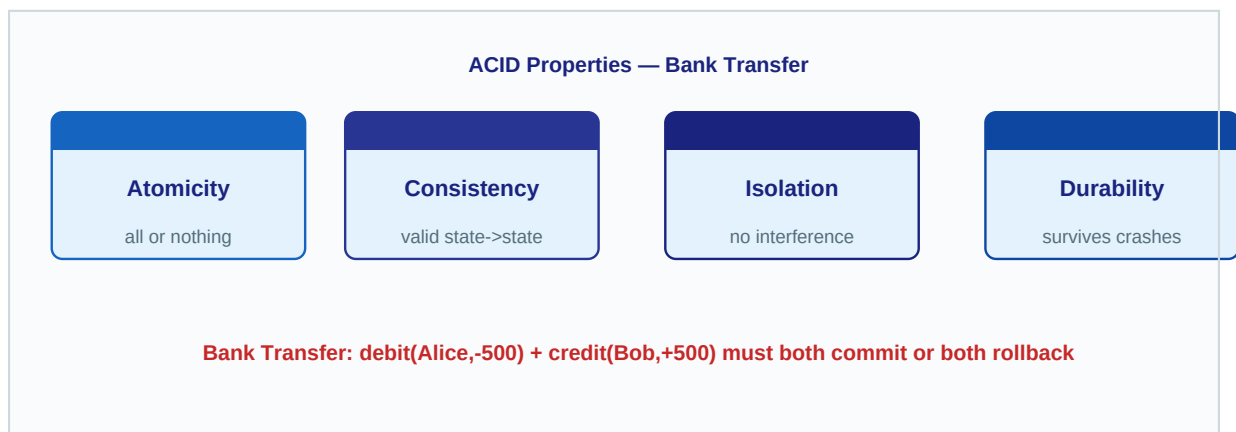


Figure A-1: ACID properties applied to a bank transfer

5. Behind the Scenes

At the JDBC level the driver sets **autoCommit=false** on the Connection. Each SQL statement is buffered in an open transaction on the database server. A final **COMMIT** (or **ROLLBACK**) flushes or discards the buffer. Spring wraps all of this inside **PlatformTransactionManager** — an abstraction that works identically whether the underlying resource is JDBC, JPA, JMS, or a NoSQL store that supports transactions.

6. Realistic Practical Example

```
@Service
public class BankService {

    @Transactional          // Spring handles begin/commit/rollback
    public void transfer(Long fromId, Long toId, BigDecimal amount) {
        Account from = accountRepo.findById(fromId).orElseThrow();
        Account to   = accountRepo.findById(toId).orElseThrow();
        from.debit(amount);
        to.credit(amount);
        // If any RuntimeException thrown above, Spring rolls back
    }
}
```

7. Common Mistakes and Misconceptions

Key Rules

- Assuming autoCommit=true is safe for multi-step operations — it is not; partial commits leave data inconsistent.
- Confusing Isolation with Atomicity — Isolation is about concurrency, Atomicity about all-or-nothing.
- Thinking @Transactional alone is enough without a proper PlatformTransactionManager bean configured.
- Forgetting that DDL statements (CREATE TABLE) auto-commit in many databases regardless of your transaction.

B @Transactional

How the annotation wires up at the most basic level

1. Plain English Explanation

@Transactional is a declarative way to say 'wrap this method in a transaction'. Spring reads the annotation at startup, creates an AOP proxy around your bean, and intercepts every call to annotated methods. You never write begin/commit/rollback yourself — Spring does it via the proxy before and after your method runs.

2. Real-World Analogy

Think of a receptionist at a bank branch. Every time a customer (caller) wants to see a teller (your service method), the receptionist first opens a transaction register, lets the teller do the work, then either stamps the register DONE or tears out the page if something went wrong. Your teller never touches the register.

3. Minimal Working Code Example

```
@Service
public class OrderService {

    @Transactional // default: propagation=REQUIRED, rollbackOn=RuntimeException
    public Order placeOrder(OrderRequest req) {
        Order o = orderRepo.save(new Order(req));
        inventoryRepo.decrementStock(req.getProductId(), req.getQty());
        return o;
        // Spring commits here if no exception; rolls back on RuntimeException
    }
}
```

4. @Transactional Proxy Flow Diagram

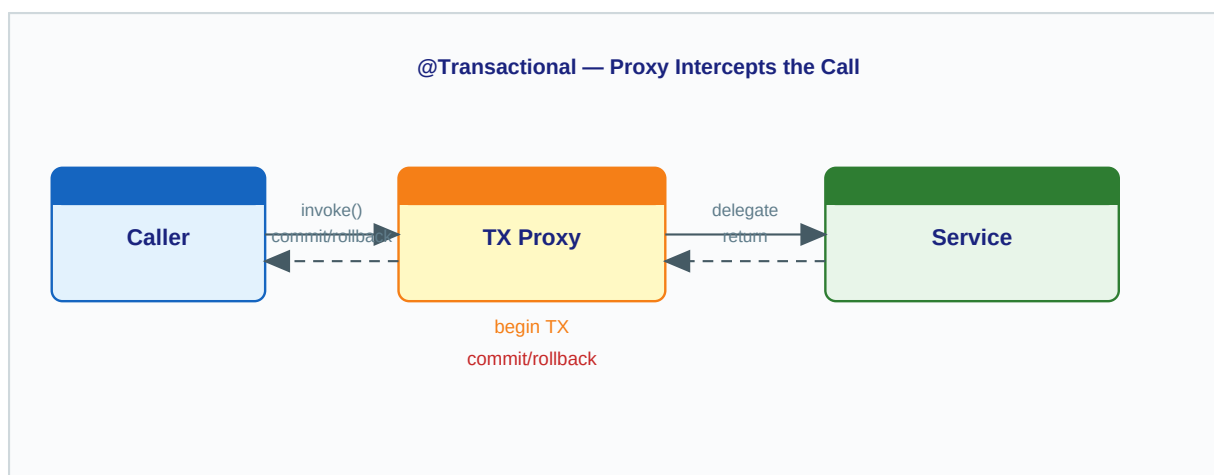


Figure B-1: Spring AOP proxy intercepts method call to manage transaction lifecycle

5. Behind the Scenes

Spring uses **@EnableTransactionManagement** (or auto-configuration in Boot) to register **InfrastructureAdvisorAutoProxyCreator**. This `BeanPostProcessor` wraps any bean that has **@Transactional** methods with a CGLIB subclass proxy (or JDK dynamic proxy for interfaces). The advice applied is **BeanFactoryTransactionAttributeSourceAdvisor**, which delegates to **TransactionInterceptor**. The attributes (propagation, isolation, rollbackFor, etc.) are parsed from the

annotation by **AnnotationTransactionAttributeSource** at startup and cached in a `ConcurrentHashMap` for fast lookup per method.

6. Realistic Practical Example

```
@Transactional(
    propagation = Propagation.REQUIRED,
    isolation    = Isolation.READ_COMMITTED,
    timeout      = 30,
    rollbackFor  = {SQLException.class, BusinessException.class},
    readOnly     = false
)
public void processPayment(Payment payment) {
    paymentRepo.save(payment);
    ledgerRepo.addEntry(payment.toLedgerEntry());
    notifyAuditLog(payment); // also participates in same TX
}
```

7. Common Mistakes and Misconceptions

Key Rules

- `@Transactional` on a private method: the proxy cannot override private methods — the annotation is silently ignored.
- Placing `@Transactional` on the implementation class when using a JDK proxy (interface-based) — use it on the interface method OR switch to CGLIB.
- Assuming `@Transactional` is active on a bean not managed by Spring (i.e., created with 'new').
- Forgetting `@EnableTransactionManagement` in a plain Spring app (Spring Boot adds it automatically).

C AOP Proxy & TransactionInterceptor

invokeWithinTransaction() — step-by-step walkthrough

1. Plain English Explanation

When you call a `@Transactional` method, Spring's proxy routes the call through `TransactionInterceptor.invoke()`. That method delegates to `TransactionAspectSupport.invokeWithinTransaction()`, which: looks up transaction attributes, asks the `PlatformTransactionManager` for a transaction (or joins an existing one), runs your method, and either commits or rolls back based on the outcome.

2. Real-World Analogy

A security guard (`TransactionInterceptor`) stands at the door of every annotated method. Before letting you in, the guard checks the rulebook (`TransactionAttributeSource`), opens a new logbook or continues an existing one (`PlatformTransactionManager`), then lets you do your work. On the way out the guard either seals the logbook (commit) or shreds it (rollback) depending on whether you triggered an alarm.

3. Minimal Working Code — annotated call trace

```
// What Spring effectively runs for every @Transactional call:
// TransactionInterceptor.invoke(MethodInvocation invocation)
// -> TransactionAspectSupport.invokeWithinTransaction(...)
// 1. txAttr = getTransactionAttributeSource().getTransactionAttribute(method, targetClass)
// 2. tm      = determineTransactionManager(txAttr)
// 3. txInfo = createTransactionIfNecessary(tm, txAttr, joinpointId)
// 4. retVal = invocation.proceed() // YOUR code runs here
// 5. if exception -> completeTransactionAfterThrowing(txInfo, ex)
// 6. else        -> commitTransactionAfterReturning(txInfo)
// 7. finally     -> cleanupTransactionInfo(txInfo)
```

4. Interceptor Call Stack Diagram

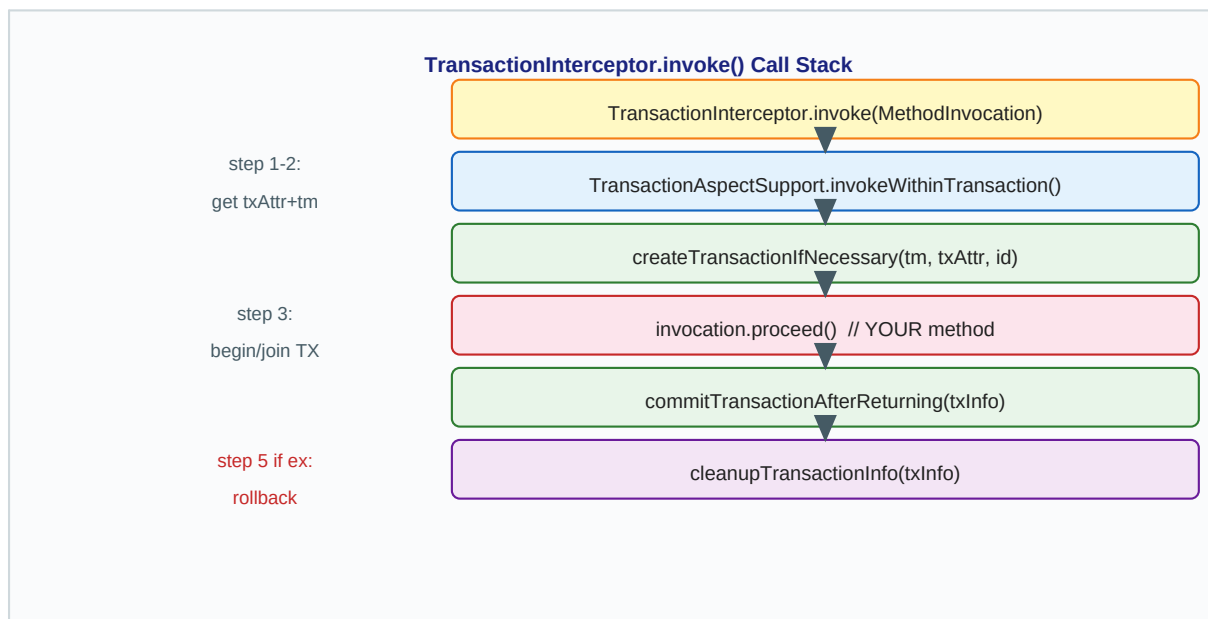


Figure C-1: `invokeWithinTransaction()` call stack inside Spring's transaction advice

5. Behind the Scenes — Key Classes

TransactionInterceptor extends **TransactionAspectSupport** and implements **MethodInterceptor**. Its **invoke()** method calls **invokeWithinTransaction()** passing the method, target class, and a callback that calls **invocation.proceed()**. The **TransactionInfo** object created by **createTransactionIfNecessary()** stores the **TransactionStatus** and is pushed onto a ThreadLocal stack (**transactionInfoHolder**), so nested calls can find the outer transaction context. After your method returns, **commitTransactionAfterReturning()** calls **tm.commit(status)**. On exception, **completeTransactionAfterThrowing()** checks the rollback rules (see Section F) and calls either **tm.rollback()** or **tm.commit()**.

6. Realistic Practical Example

```
// Enable transaction management (or use Spring Boot auto-config)
@Configuration
@EnableTransactionManagement
public class TxConfig {
    @Bean
    public PlatformTransactionManager transactionManager(DataSource ds) {
        return new DataSourceTransactionManager(ds);
        // For JPA use: new JpaTransactionManager(entityManagerFactory)
    }
}
// Now every @Transactional bean is wrapped by TransactionInterceptor automatically
```

7. Common Mistakes and Misconceptions

Key Rules

- Thinking Spring uses AspectJ LTW by default — it uses CGLIB/JDK proxies (Spring proxy mode), not compile-time weaving.
- Expecting TransactionInterceptor to catch Errors (OutOfMemoryError) — it only handles Throwable subclasses matching rollback rules.
- Not realising TransactionInfo is ThreadLocal — moving work to a new thread loses the transaction context.
- Calling tm.commit() manually after @Transactional already committed — results in 'transaction already completed' IllegalStateException.

D Propagation

REQUIRED, REQUIRES_NEW, NESTED — with timeline diagrams

1. Plain English Explanation

Propagation controls what happens when a `@Transactional` method is called while a transaction is already active on the current thread. Spring offers 7 modes; the three most important are: **REQUIRED** (join or create), **REQUIRES_NEW** (always create, suspend the existing one), and **NESTED** (create a savepoint inside the existing one).

Key Propagation Modes

Mode	Existing TX?	Behaviour	Use-case
REQUIRED (default)	Yes	Join it	Standard service methods
REQUIRED	No	Create new	Standard service methods
REQUIRES_NEW	Yes	Suspend outer, create new	Audit log that must commit regardless
REQUIRES_NEW	No	Create new	Audit log
NESTED	Yes	Savepoint in outer TX	Partial rollback of sub-operation
NESTED	No	Create new	Partial rollback of sub-operation
SUPPORTS	Yes	Join it	Read-only queries
NOT_SUPPORTED	Yes	Suspend, run non-TX	Caching / non-TX calls
NEVER	Yes	Throw exception	Enforce no-TX zone
MANDATORY	No	Throw exception	Must be called from TX

2. Real-World Analogy

REQUIRED: You walk into an open meeting room — you join the meeting. **REQUIRES_NEW**: You lock the current meeting room, go to a different room, hold your private meeting, come back when done. The original meeting was paused while you were away. **NESTED**: During the meeting you open a sub-agenda item; if that sub-item fails you rewind only to just before it started — the main meeting continues.

3. Minimal Working Code Example

```
@Service public class OrderService {
    @Autowired AuditService audit;

    @Transactional(propagation = Propagation.REQUIRED)
    public void placeOrder(Order o) {
        orderRepo.save(o);
        audit.log("ORDER_PLACED", o.getId()); // REQUIRES_NEW -> separate TX
        // If audit.log() fails, its TX rolls back but this TX still commits
    }
}
```



```

    }
}
@Service public class AuditService {
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void log(String event, Long id) {
        auditRepo.save(new AuditEntry(event, id)); // own TX
    }
}

```

4. Propagation Timeline Diagram

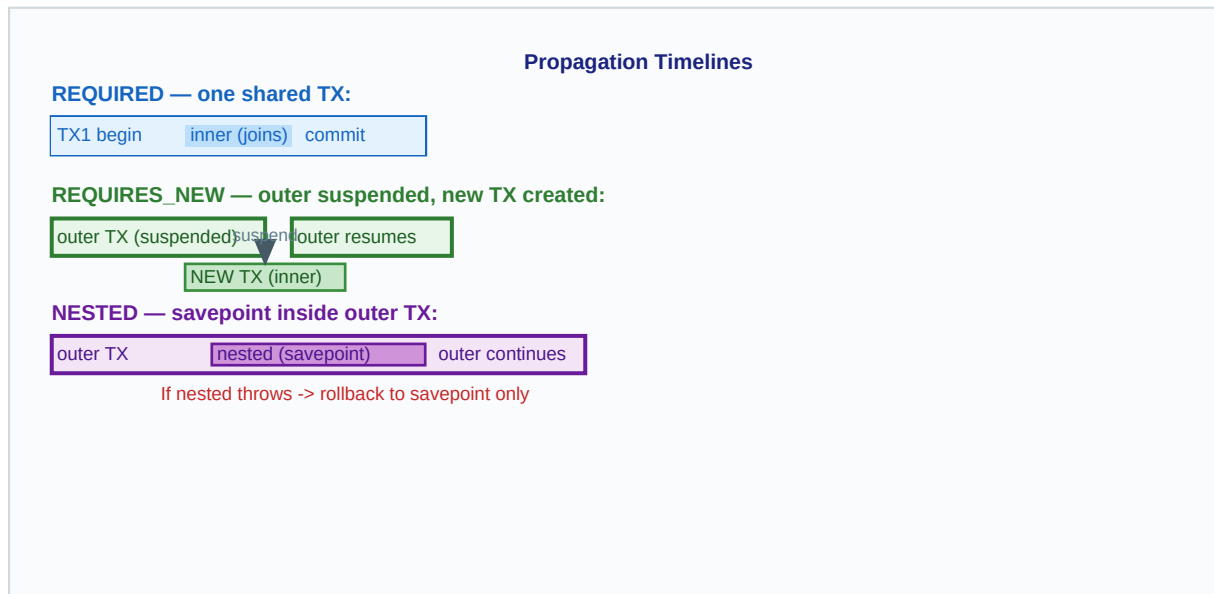


Figure D-1: Transaction boundaries for REQUIRED, REQUIRES_NEW, and NESTED propagation

5. Behind the Scenes

In **AbstractPlatformTransactionManager.getTransaction()** Spring checks the current thread's transaction via **isExistingTransaction()**. If one exists and propagation is **REQUIRES_NEW**, **handleExistingTransaction()** calls **suspend()** which unbinds the current DataSource-to-Connection mapping from **TransactionSynchronizationManager** and stores it in a **SuspendedResourcesHolder**. A brand-new Connection is obtained and bound. When the inner method finishes, **resume()** restores the holder. For **NESTED**, Spring calls **connection.setSavepoint()** directly — the DB's savepoint mechanism does the work.

6. Realistic Practical Example

```

@Transactional(propagation = Propagation.REQUIRES_NEW)
public void writeAudit(String message) {
    // Gets its OWN DB connection from the pool
    // SuspendedResourcesHolder stores outer connection while this runs
    auditRepo.save(new Audit(message, Instant.now()));
    // Commits independently – outer TX failure won't roll back this
}
// Outer TX failure does NOT undo audit entries – great for compliance logs

```

7. Common Mistakes and Misconceptions

Key Rules

- `REQUIRES_NEW` still shares the same DB if isolation demands — it uses a `DIFFERENT` connection, not a different database.
- Using `NESTED` with a JPA `EntityManager` that does not support savepoints — always test with your actual DB driver.
- Expecting `REQUIRES_NEW` inner TX rollback to roll back the outer TX — they are fully independent.
- Forgetting that `NESTED` is not supported by all `PlatformTransactionManager` implementations (e.g., `JtaTransactionManager`).

E

Isolation Levels

Concurrency problems: dirty read, phantom read, non-repeatable read

1. Plain English Explanation

Isolation controls how much one transaction sees the uncommitted changes of another concurrent transaction. Higher isolation = fewer anomalies = more locking = less throughput. The four standard levels each prevent a different class of concurrency bug.

Level	Dirty Read	Non-Repeatable Read	Phantom Read	Typical Use
READ_UNCOMMITTED	Possible	Possible	Possible	Rarely used; analytics shortcuts
READ_COMMITTED	Prevented	Possible	Possible	PostgreSQL/Oracle default
REPEATABLE_READ	Prevented	Prevented	Possible	MySQL InnoDB default
SERIALIZABLE	Prevented	Prevented	Prevented	Financial / high-integrity ops

2. Real-World Analogy

Dirty Read: You see a colleague's draft email before they decide to send or delete it. **Non-Repeatable Read:** You read a price, step away, come back and re-read — the price changed. **Phantom Read:** You count 5 items in a list, do some work, count again — now there are 6 because someone inserted one while you worked.

3. Minimal Working Code Example

```
@Transactional(isolation = Isolation.REPEATABLE_READ)
public ProductReport buildReport(Long productId) {
    // First read
    Product p = productRepo.findById(productId).orElseThrow();
    processData(p);
    // Second read – guaranteed same data even if another TX updated it
    Product p2 = productRepo.findById(productId).orElseThrow();
    return new ProductReport(p, p2);
}
```

4. Concurrency Anomaly Diagrams

Isolation Level Anomalies — Timeline

Dirty Read (READ_UNCOMMITTED):

TX1: UPDATE balance=500 (not committed)
TX2: SELECT balance -> sees 500 (WRONG!)
TX1: ROLLBACK -> balance back to 300
TX2 used stale data: 500 was never real

Non-Repeatable Read (READ_COMMITTED):

TX2: SELECT price -> 100
TX1: UPDATE price=150; COMMIT
TX2: SELECT price again -> 150 (changed!)

Phantom Read (REPEATABLE_READ):

TX2: SELECT COUNT(*) WHERE dept='IT' -> 5
TX1: INSERT new employee in IT; COMMIT
TX2: SELECT COUNT(*) again -> 6 (phantom!)

Figure E-1: Dirty read, non-repeatable read, and phantom read anomalies

5. Behind the Scenes

Spring maps the Isolation enum to JDBC's **Connection.setTransactionIsolation(int)** inside **DataSourceTransactionManager.doBegin()** (or **JpaTransactionManager.doBegin()**). The integer constants are defined on **java.sql.Connection**. If isolation is DEFAULT, the driver's current default is used unchanged. Note: changing isolation mid-connection can be unsupported by some drivers; Spring always applies isolation at the start of a new transaction.

6. Realistic Practical Example

```
// Payment confirmation requires no phantom reads
@Transactional(isolation = Isolation.SERIALIZABLE)
public ConfirmationReceipt confirmPayment(Long paymentId) {
    Payment p = paymentRepo.findById(paymentId).orElseThrow();
    if (p.getStatus() != PENDING) throw new IllegalStateException("Already processed");
    p.setStatus(CONFIRMED);
    return receiptRepo.save(new ConfirmationReceipt(p));
}
```

7. Common Mistakes and Misconceptions

Key Rules

- Using SERIALIZABLE everywhere 'to be safe' — serializable locks rows/ranges aggressively, killing throughput under load.
- Assuming READ_COMMITTED prevents all anomalies — it only prevents dirty reads; non-repeatable and phantom reads remain.
- Mixing isolation levels without understanding the DB default — MySQL InnoDB defaults to REPEATABLE_READ; PostgreSQL to READ_COMMITTED.
- Forgetting that JPA second-level cache can bypass isolation — cached entities may be stale regardless of DB isolation.

F Rollback Rules

Default RuntimeException-only behaviour, rollbackFor, noRollbackFor

1. Plain English Explanation

By default Spring only rolls back on **unchecked exceptions** (RuntimeException and its subclasses) and **Errors**. Checked exceptions (IOException, SQLException, etc.) cause a commit unless you explicitly configure rollbackFor. You can also tell Spring NOT to rollback on specific exceptions with noRollbackFor.

2. Real-World Analogy

Your bank teller has a rulebook: if the customer shouts (RuntimeException) the transaction is cancelled. But if the printer jams (checked IOException) the teller assumes it is recoverable and commits the transaction anyway — unless the rulebook explicitly says 'printer jam = cancel transaction' (rollbackFor=IOException.class).

3. Minimal Working Code Example

```
// Default: rolls back only on RuntimeException / Error
@Transactional
public void defaultBehaviour() throws IOException {
    repo.save(entity);
    throw new IOException("checked"); // COMMITS despite exception!
}

// Explicit: also rollback on checked exceptions
@Transactional(rollbackFor = IOException.class)
public void rollbackOnChecked() throws IOException {
    repo.save(entity);
    throw new IOException("checked"); // NOW rolls back
}

@Transactional(noRollbackFor = OptimisticLockException.class)
public void ignoreOptimisticLock() { /* retries handled at caller */ }
```

4. Rollback Decision Diagram

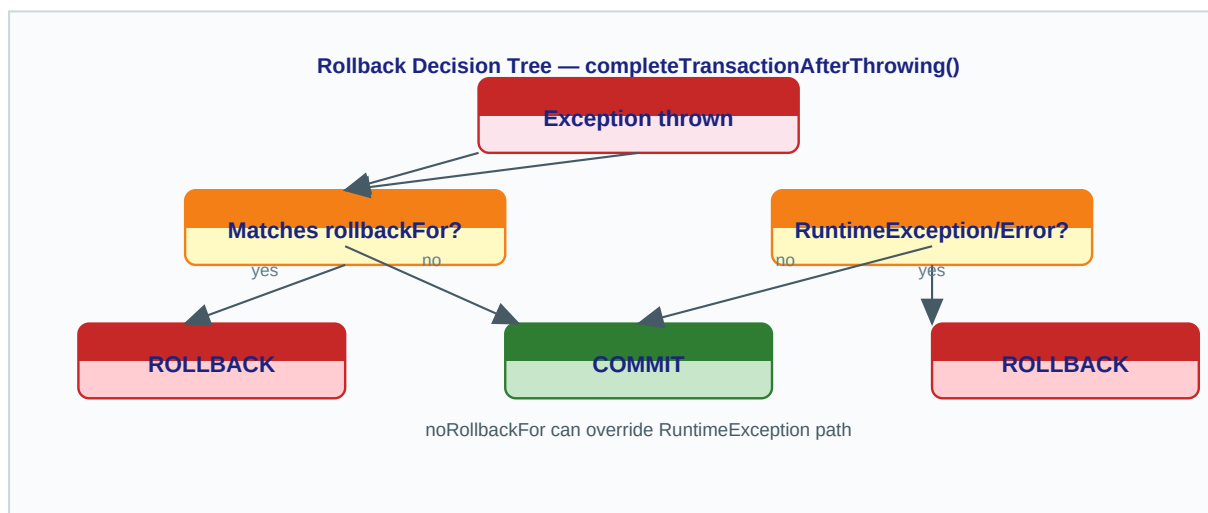


Figure F-1: completeTransactionAfterThrowing() rollback decision logic

5. Behind the Scenes

TransactionAspectSupport.completeTransactionAfterThrowing() calls **txInfo.transactionAttribute.rollbackOn(ex)**. The attribute is a **RuleBasedTransactionAttribute** that holds a list of **RollbackRuleAttribute** objects (each wrapping a class). The method iterates the list finding the most-specific match (closest superclass). A **NoRollbackRuleAttribute** wins over a plain **RollbackRuleAttribute** at the same depth. If no custom rule matches, it falls back to **DefaultTransactionAttribute.rollbackOn()** which checks **ex instanceof RuntimeException || ex instanceof Error**.

6. Realistic Practical Example

```
@Transactional(
    rollbackFor = {PaymentGatewayException.class, IOException.class},
    noRollbackFor = {OptimisticLockException.class}
)
public Receipt charge(Order order) throws PaymentGatewayException, IOException {
    Receipt r = paymentGateway.charge(order.totalAmount()); // may throw
    orderRepo.save(order.withStatus(CHARGED));
    return r;
    // OptimisticLockException -> COMMIT (caller retries)
    // IOException             -> ROLLBACK
    // PaymentGatewayException -> ROLLBACK
}
```

7. Common Mistakes and Misconceptions

Key Rules

- Catching a **RuntimeException** inside the **@Transactional** method and re-throwing a checked one — Spring never sees the original **RuntimeException**, so it may commit.
- Expecting **rollbackFor** to work transitively on subclasses — it does, via **isAssignableFrom()** logic in **RollbackRuleAttribute**.
- Using **rollbackFor=Exception.class** to catch everything, then being surprised that checked business exceptions also roll back.
- Not knowing that calling **setRollbackOnly()** on the **TransactionStatus** is the programmatic alternative to throwing an exception.

G

The Self-Invocation Trap

Why same-class `@Transactional` calls bypass the proxy

1. Plain English Explanation

Spring's transaction management works by wrapping your bean in an AOP proxy. When an **external caller** invokes a method, the call passes through the proxy. But when a method inside the **same class** calls another method in that class using **this.someMethod()**, the call goes directly to the real object — the proxy is bypassed. Any `@Transactional` annotation on that inner method is completely ignored.

The same applies to **private methods**: CGLIB proxies cannot override private methods, so `@Transactional` on a private method is a silent no-op.

2. Real-World Analogy

The security guard (proxy) only intercepts visitors coming through the front door. If a teller inside the building walks down the hallway to talk to a colleague (internal `this.method()` call), they go straight to each other — no guard, no logbook check, no transaction check.

3. Minimal Working Code Example — Bug and Fix

```
// BUG: self-invocation bypasses proxy
@Service public class PaymentService {
    @Transactional
    public void outer() {
        inner(); // calls this.inner() -> proxy NOT involved!
    }
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void inner() { /* REQUIRES_NEW is silently ignored */ }
}

// FIX 1: inject the bean into itself (Spring supports this with @Lazy)
@Service public class PaymentService {
    @Autowired @Lazy PaymentService self;
    public void outer() { self.inner(); } // goes through proxy
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void inner() { /* now works */ }
}
```

4. Self-Invocation vs External Call Diagram

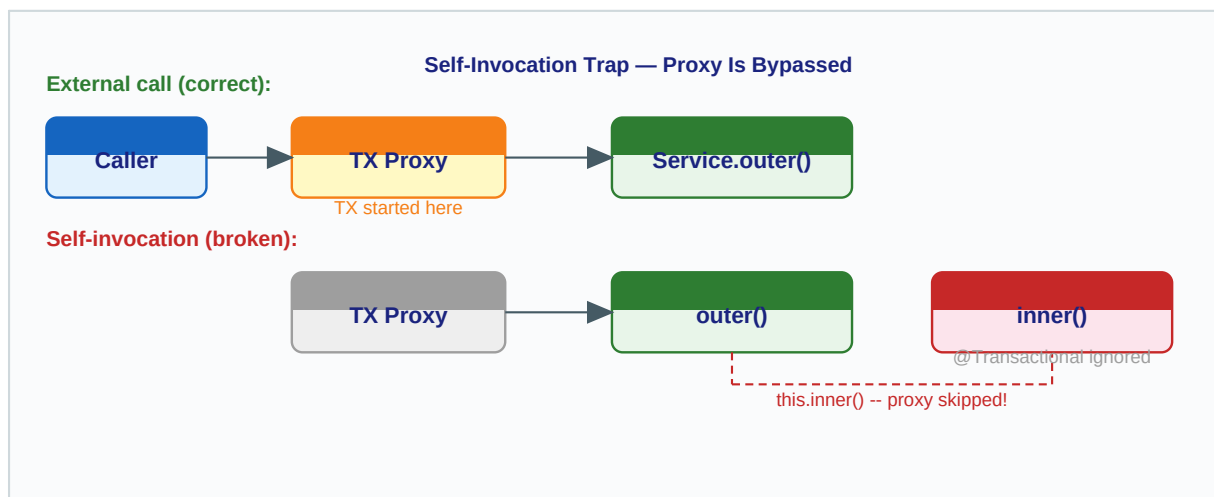


Figure G-1: Self-invocation bypasses the AOP proxy — @Transactional on inner() has no effect

5. Behind the Scenes

CGLIB creates a subclass of your bean at runtime. The subclass overrides each public non-final method and routes calls through the advice chain. **Private methods cannot be overridden in a subclass** — Java forbids it — so CGLIB cannot intercept them. For same-class calls, Java resolves **this.method()** directly to the real instance's vtable, not to the proxy subclass's override. The proxy reference was never involved. The only solutions are: (1) inject the proxy into itself via @Autowired @Lazy self; (2) split the class; (3) use AspectJ compile-time or load-time weaving which operates on bytecode and does intercept this-calls.

6. Realistic Practical Example — Fix with Split Class

```
// Preferred fix: extract inner logic into a separate Spring bean
@Service public class OrderService {
    @Autowired AuditWriter auditWriter;

    @Transactional
    public void createOrder(Order o) {
        orderRepo.save(o);
        auditWriter.writeAudit("ORDER_CREATED", o.getId()); // external call -> proxy works
    }
}
@Service public class AuditWriter {
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void writeAudit(String event, Long id) {
        auditRepo.save(new AuditEntry(event, id));
    }
}
```

7. Common Mistakes and Misconceptions

Key Rules

- Adding @Transactional to private helper methods expecting them to start transactions — silently does nothing with proxy-based AOP.
- Thinking final methods work — CGLIB cannot override final methods either, same result as private.
- Using AopContext.currentProxy() hack without enabling exposeProxy=true on @EnableTransactionManagement.
- Believing the issue is fixed just by adding @Transactional to outer() — the inner annotation is still bypassed.

H

Transaction Synchronisation

ThreadLocal binding — TransactionSynchronizationManager internals

1. Plain English Explanation

TransactionSynchronizationManager (TSM) is the central registry that binds resources (DB connections, Hibernate Sessions) to the current thread using ThreadLocal variables. This is how a single @Transactional call can span multiple repository calls, JdbcTemplate queries, or EntityManager operations — they all find the same connection by looking it up from the thread-local map.

2. Real-World Analogy

Every employee (thread) has a personal locker (ThreadLocal). When a transaction starts the manager puts a key (connection) in that locker. All tools (repositories, templates) owned by that employee check the locker first before asking the pool for a new key. When the transaction ends the key is removed from the locker and returned to the pool.

3. Minimal Working Code Example

```
// Spring does this for you; shown here for illustration:  
// 1. Transaction manager obtains connection and binds it  
TransactionSynchronizationManager.bindResource(dataSource, connectionHolder);  
TransactionSynchronizationManager.setActualTransactionActive(true);  
  
// 2. Later, JdbcTemplate fetches the SAME connection from the thread  
ConnectionHolder holder =  
    (ConnectionHolder) TransactionSynchronizationManager.getResource(dataSource);  
Connection conn = holder.getConnection();  
  
// 3. After commit/rollback, unbind  
TransactionSynchronizationManager.unbindResource(dataSource);
```

4. ThreadLocal Binding Diagram

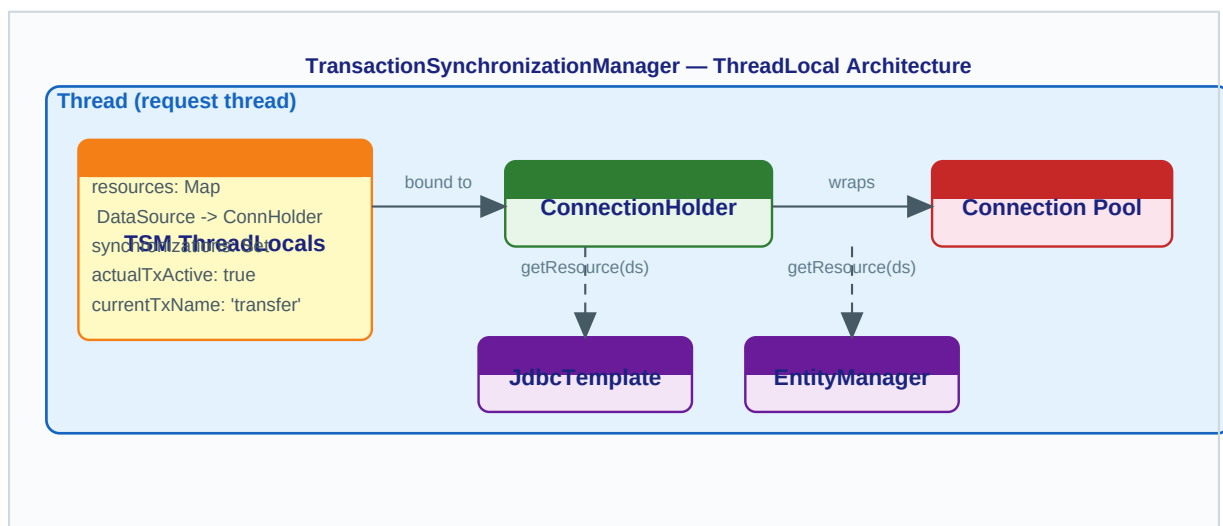


Figure H-1: TSM stores DataSource->ConnectionHolder in ThreadLocal; all consumers share the same connection

5. Behind the Scenes

TransactionSynchronizationManager holds several **ThreadLocal** fields: **resources** (IdentityHashMap of resource key -> holder), **synchronizations** (LinkedHashSet of TransactionSynchronization callbacks), **currentTransactionName**, **currentTransactionReadOnly**, **currentTransactionIsolationLevel**, and **actualTransactionActive**. When **DataSourceTransactionManager.doBegin()** opens a connection it calls **bindResource(dataSource, new ConnectionHolder(con))**. **DataSourceUtils.getConnection()** (called by JdbcTemplate) first checks **TransactionSynchronizationManager.getResource(dataSource)** — if found it reuses the bound connection; otherwise it opens a new one (non-transactional).

6. Realistic Practical Example — Custom Synchronisation

```
// Register a callback to run AFTER the transaction commits
TransactionSynchronizationManager.registerSynchronization(
    new TransactionSynchronizationAdapter() {
        @Override
        public void afterCommit() {
            // Send Kafka message ONLY after DB row is committed
            kafkaTemplate.send("orders", orderId.toString());
        }
    }
);
```

7. Common Mistakes and Misconceptions

Key Rules

- Spawning a new thread inside `@Transactional` and expecting it to share the transaction — new threads have empty `ThreadLocals`.
- Using `@Async` inside `@Transactional` — the async thread starts a new context; the calling transaction is NOT propagated.
- Directly calling `DataSource.getConnection()` instead of `DataSourceUtils.getConnection()` — bypasses TSM, gets a fresh connection outside the transaction.
- Forgetting to unbind resources in custom transaction managers — causes 'resource already bound' errors on the next request to the same thread (pooled threads).

1. Plain English Explanation

TransactionTemplate lets you manage transactions in code rather than with annotations. You provide a callback (lambda) and Spring wraps it in a transaction. Use it when: (a) transaction boundaries are dynamic/conditional, (b) you are writing code that cannot use annotations (e.g. in a constructor or utility class not managed by Spring), or (c) you need very fine-grained control over partial commits.

2. Real-World Analogy

@Transactional is like a sign on the door: 'all work in this room is transactional'. TransactionTemplate is like calling security manually: 'hey, wrap only these three specific lines in a transaction, not the whole method'.

3. Minimal Working Code Example

```
@Service
public class BatchProcessor {
    private final TransactionTemplate txTemplate;

    public BatchProcessor(PlatformTransactionManager tm) {
        this.txTemplate = new TransactionTemplate(tm);
        this.txTemplate.setPropagationBehavior(TransactionDefinition.PROPROPAGATION_REQUIRES_NEW);
        this.txTemplate.setIsolationLevel(TransactionDefinition.ISOLATION_READ_COMMITTED);
    }

    public void processBatch(List<Record> records) {
        for (Record r : records) {
            txTemplate.execute(status -> {        // one TX per record
                repo.save(r);
                return null; // return null -> commit; throw -> rollback
            });
        }
    }
}
```

4. TransactionTemplate Flow Diagram

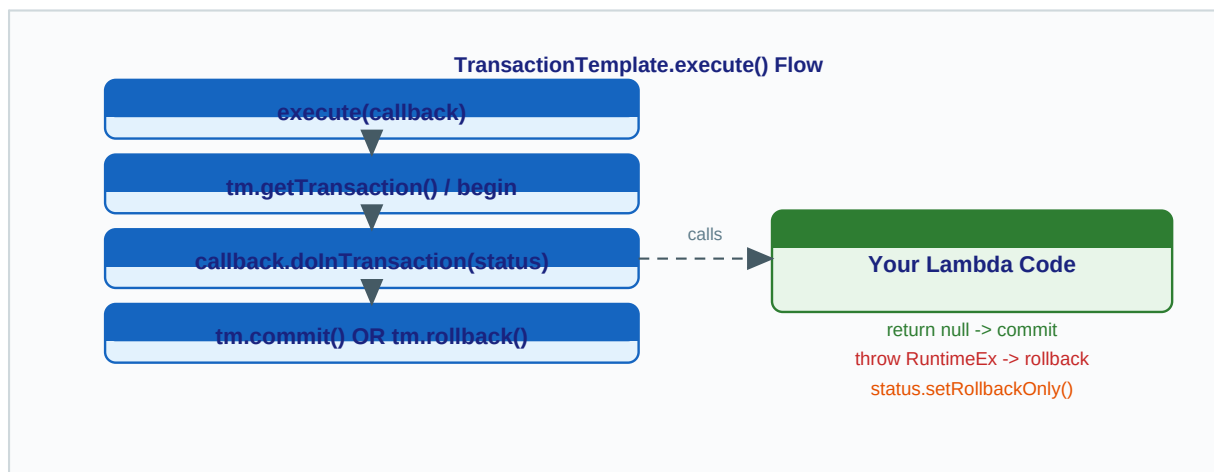


Figure I-1: TransactionTemplate.execute() manages the full transaction lifecycle around your lambda

5. Behind the Scenes

TransactionTemplate extends **DefaultTransactionDefinition** and implements **TransactionOperations**. Its **execute()** method calls **platformTransactionManager.getTransaction(this)** to obtain a **TransactionStatus**, invokes **callback.doInTransaction(status)**, and then commits. If a **RuntimeException** escapes the callback, or if **status.setRollbackOnly()** was called, it rolls back. The template also handles **TransactionException** wrapping. Internally it goes through the same **AbstractPlatformTransactionManager** code path as the declarative approach — no magic, just direct method calls instead of AOP advice.

6. Realistic Practical Example

```
// Conditional transaction: only wrap in TX when record count > threshold
public void conditionalBatch(List<Item> items) {
    if (items.size() > 100) {
        txTemplate.execute(s -> { bulkSave(items); return null; });
    } else {
        // Small batch: each save in its own auto-commit statement
        items.forEach(repo::save);
    }
}

// Also useful: manual rollback without throwing
txTemplate.execute(status -> {
    Order o = orderRepo.findById(id).orElseThrow();
    if (o.isCancelled()) { status.setRollbackOnly(); return null; }
    process(o);
    return o;
});
```

7. Common Mistakes and Misconceptions

Key Rules

- Creating a new **TransactionTemplate** for every call — it is thread-safe after construction, so create once and reuse.
- Returning a value from **execute()** but expecting a rollback — returning any value (including null) causes a commit unless **setRollbackOnly()** was called.
- Using **TransactionTemplate** inside a **@Transactional** method expecting it to create a new TX — it follows propagation rules just like annotations; defaults to **REQUIRED** (joins existing).
- Forgetting to set propagation on the template when **REQUIRES_NEW** behaviour is needed — default is **REQUIRED**.

1. Plain English Explanation

When you mark a transaction **readOnly=true**, Spring and the underlying framework apply two key optimisations: (1) the JDBC connection is flagged read-only, allowing the database to skip certain lock acquisitions; and (2) the JPA/Hibernate session's flush mode is set to MANUAL, meaning Hibernate will not scan entities for dirty changes before committing — saving CPU and memory on large result sets.

2. Real-World Analogy

Reading a book from a library (read-only) vs. borrowing it to annotate (read-write). The librarian (Hibernate) does not need to check whether you wrote in the margins when you return it from a read-only session — no dirty-check required. The database librarian (RDBMS) also does not bother acquiring row-level write locks.

3. Minimal Working Code Example

```
@Transactional(readOnly = true)
public List<ProductSummary> listProducts(Pageable page) {
    // Hibernate FlushMode = MANUAL -> no dirty check on commit
    // Connection.setReadOnly(true) -> DB may skip write locks
    // Some DBs route to read replica -> lower latency
    return productRepo.findAll(page)
        .map(ProductSummary::from)
        .toList();
}
```

4. Read-Only Optimisation Diagram

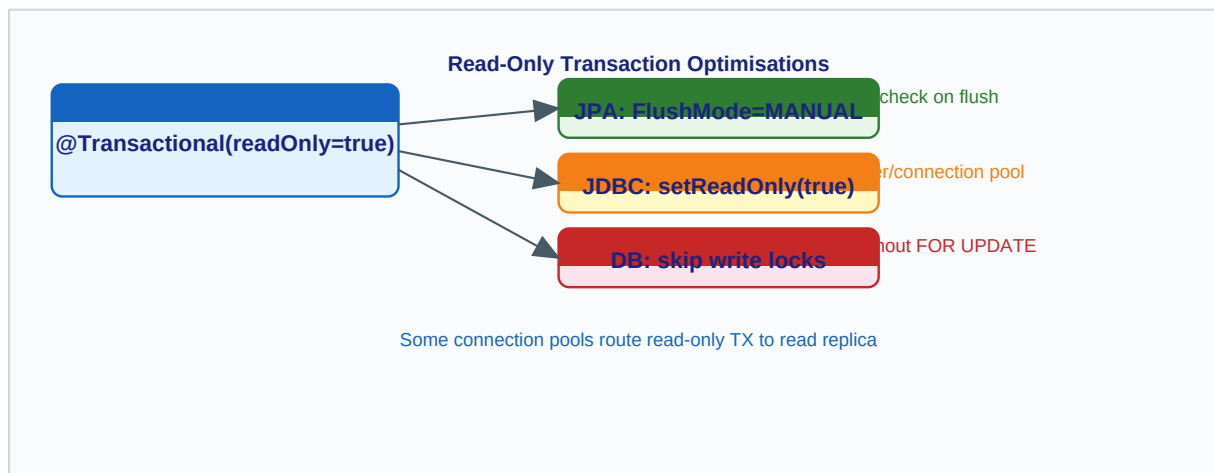


Figure J-1: readOnly=true triggers three distinct optimisations at different layers

5. Behind the Scenes

In **JpaTransactionManager.doBegin()** Spring checks **definition.isReadOnly()**. If true, it calls **session.setFlushMode(FlushModeType.COMMIT)** on the EntityManager (which maps to Hibernate's **FlushMode.MANUAL** in practice for Spring). This prevents Hibernate's dirty-checking mechanism (**DefaultFlushEventListener**) from firing before commit. Additionally, **DataSourceUtils.prepareConnectionForTransaction()** calls **con.setReadOnly(true)** on the JDBC

Connection. Many databases (PostgreSQL, MySQL) interpret this hint to avoid acquiring shared/exclusive locks. Some connection pools (HikariCP with **readOnly** datasource routing) redirect the connection to a read replica entirely.

6. Realistic Practical Example

```
// Repository layer – mark all query methods read-only at class level
@Transactional(readOnly = true)
public interface ReportRepository extends JpaRepository<Report, Long> {
    List<Report> findByCreatedAtBetween(LocalDate from, LocalDate to);
    @Query("SELECT r FROM Report r WHERE r.status = 'PUBLISHED'")
    List<Report> findPublished();
}
// Service overrides for write operations
@Transactional // readOnly=false overrides class-level readOnly=true
public Report publish(Long id) { ... }
```

7. Common Mistakes and Misconceptions

Key Rules

- Assuming readOnly=true prevents writes at the Java level — it does NOT; you can still call save(). The DB may or may not reject it depending on driver and DB.
- Expecting readOnly to always route to a replica — this requires additional infrastructure (AbstractRoutingDataSource) and is not automatic.
- Setting readOnly=true on a method that calls a write repository — the flush mode change means your writes may never reach the DB (MANUAL flush = you must flush explicitly).
- Forgetting that FlushMode.MANUAL is a Hibernate detail — with plain JDBC there is no dirty check to skip; readOnly just sets the connection flag.

K

JOIN FETCH + Pagination Warning

HHH90003004 — full table loaded in memory + correct fixes

1. Plain English Explanation

When you write a JPQL query with **JOIN FETCH** on a collection relationship and combine it with **Pageable** (setMaxResults / setFirstResult), Hibernate cannot translate that into a SQL LIMIT at the database level. The reason: JOIN FETCH produces duplicate rows per collection element — if an Order has 3 items, the JOIN produces 3 rows for that one Order. Applying LIMIT 10 in SQL could cut an Order's items mid-way. Hibernate's response: load ALL rows into memory and apply the page in Java. This is the **HHH90003004** warning and can silently OOM your application.

2. Real-World Analogy

Imagine asking a librarian for 'the first 5 books along with all their chapters listed'. The librarian fetches every single book-chapter combination (thousands of rows), lays them all on the table, then manually counts out 5 books. That table is your heap.

3. The BAD Code — What Causes the Problem

```
// BAD: JOIN FETCH + Pageable -> HHH90003004 warning, full table in memory
public interface OrderRepository extends JpaRepository<Order, Long> {

    @Query("SELECT o FROM Order o JOIN FETCH o.items WHERE o.status = :status")
    Page<Order> findByStatusWithItems(
        @Param("status") String status,
        Pageable pageable // setMaxResults applied IN JAVA, not SQL
    );
}

// Hibernate log: "HHH90003004: firstResult/maxResults specified with collection
//                fetch; applying in memory!"
// With 1 million orders x 10 items = 10 million rows loaded into heap
```

4. Why Hibernate Cannot Push LIMIT to SQL — Diagram

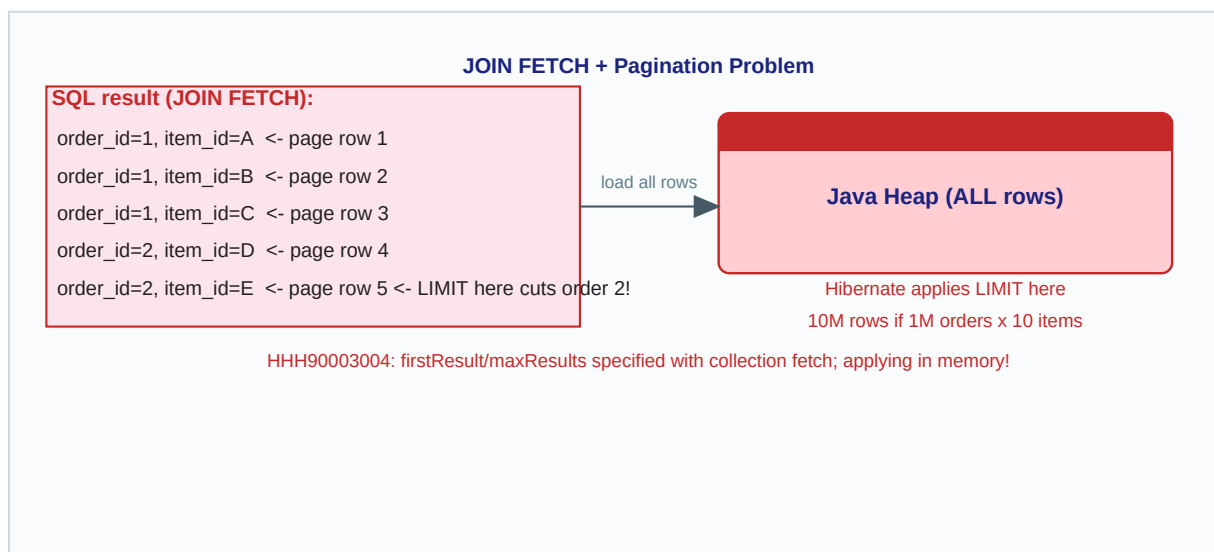


Figure K-1: JOIN FETCH forces Hibernate to load entire result set; LIMIT is applied in Java heap

5. Behind the Scenes — Why Hibernate Does This

When Hibernate processes a **CollectionFetch** alongside **setMaxResults()**, it detects the conflict in **QueryParameters** and sets a flag. The **AbstractQueryImpl** replaces the SQL-level **LIMIT/OFFSET** with in-memory limiting inside **ScrollableResultsImpl** or the row-processing loop. It logs the **HHH90003004** warning via **QueryLogger**. The root cause: a single Order entity is represented by multiple JDBC rows (one per item). Applying **LIMIT 10** in SQL could return rows 1-10 of the result set which might correspond to only 2 Orders with partial item lists — Hibernate would be forced to return malformed entity graphs.

6. The CORRECT Fix — Option A: @BatchSize

```
// FIX A: @BatchSize – load items lazily in batches after paged Order query
@Entity public class Order {
    @OneToMany(mappedBy = "order", fetch = FetchType.LAZY)
    @BatchSize(size = 20) // Hibernate loads items in IN (...) queries of 20
    private List<OrderItem> items;
}

// Repository: plain query without JOIN FETCH -- pagination works in SQL
public interface OrderRepository extends JpaRepository<Order, Long> {
    @Query("SELECT o FROM Order o WHERE o.status = :status")
    Page<Order> findByStatus(@Param("status") String status, Pageable pageable);
}

// Spring Data generates: SELECT * FROM orders WHERE status=? LIMIT ? OFFSET ?
// Then separately: SELECT * FROM order_items WHERE order_id IN (1,2,3,...,20)
```

Fix Option B: @EntityGraph with separate count query

```
// FIX B: @EntityGraph + countQuery – eager graph but SQL pagination
public interface OrderRepository extends JpaRepository<Order, Long> {

    @EntityGraph(attributePaths = {"items"})
    @Query(
        value = "SELECT o FROM Order o WHERE o.status = :status",
        countQuery = "SELECT COUNT(o) FROM Order o WHERE o.status = :status"
    )
    Page<Order> findByStatusWithGraph(
        @Param("status") String status,
        Pageable pageable
    );
}

// Spring Data uses countQuery for the count SELECT (no JOIN needed)
// Main query: Hibernate generates 2 SQL statements -- SELECT orders LIMIT/OFFSET
//              then SELECT items WHERE order_id IN (ids from page)
// No HHH90003004, no full table scan, correct pagination
```

4b. Fix Comparison Diagram

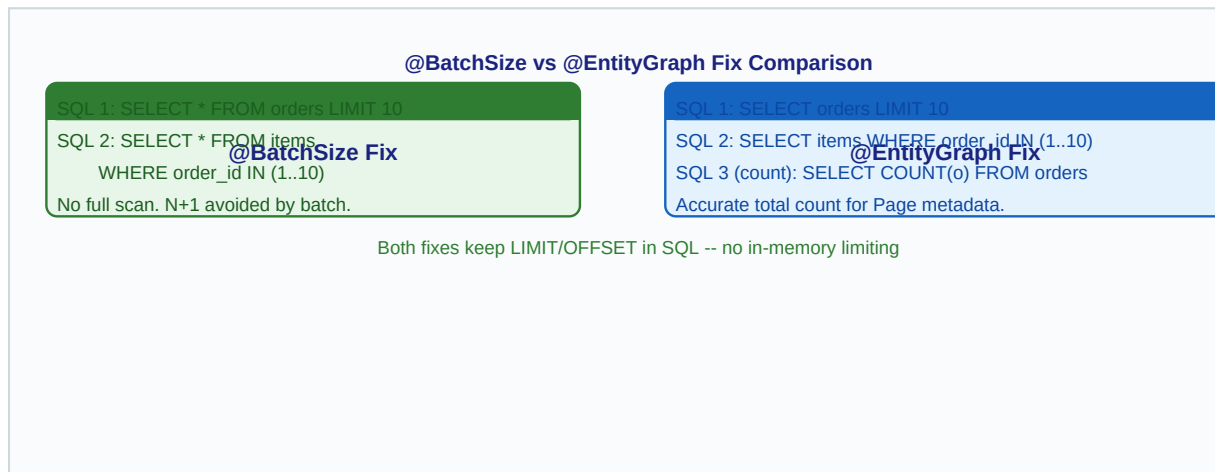


Figure K-2: Both fixes push SQL LIMIT to the database; items are fetched in separate IN() queries

7. Common Mistakes and Misconceptions

Key Rules

- Not noticing HHH90003004 in logs and deploying to production — add log monitoring for this warning in CI.
- Using @EntityGraph without a countQuery — Spring Data generates the same JOIN FETCH for the count query, which also loads all items.
- Thinking DISTINCT in JPQL fixes it — SELECT DISTINCT o FROM Order o JOIN FETCH o.items still has the same pagination conflict with Hibernate.
- Using @BatchSize(size=1) — defeats the purpose; use 20-50 matching your typical page size.
- Assuming this is a Hibernate bug — it is a fundamental SQL/relational model limitation; JOIN on one-to-many duplicates parent rows.

Topic	Key Class / Attribute	Default / Gotcha
@Transactional	TransactionInterceptor	CGLIB proxy; private methods ignored
Propagation	AbstractPlatformTransactionManager	Default: REQUIRED (join or create)
REQUIRES_NEW suspend	SuspendedResourcesHolder	New Connection, outer suspended
NESTED	connection.setSavepoint()	Not all TMs support it
Isolation	Connection.setTransactionIsolation()	DEFAULT uses DB default
Rollback default	DefaultTransactionAttribute	RuntimeException + Error only
rollbackFor checked	RuleBasedTransactionAttribute	Most-specific rule wins
Self-invocation	CGLIB proxy (this. call)	Annotation silently ignored
ThreadLocal binding	TransactionSynchronizationManager	bindResource(ds, connHolder)
TransactionTemplate	PlatformTransactionManager	setRollbackOnly() for manual rollback
readOnly=true	JpaTransactionManager	FlushMode.MANUAL + setReadOnly(true)
JOIN FETCH + Page	HHH90003004 warning	Full table in heap; use @BatchSize

Internal Method Call Chain

```
// Full call chain from @Transactional annotation to commit:
// 1. CGLIB proxy.method()
// 2. -> TransactionInterceptor.invoke(MethodInvocation)
// 3. -> TransactionAspectSupport.invokeWithinTransaction(method, class, callback)
// 4. -> AnnotationTransactionAttributeSource.getTransactionAttribute(method, class)
// 5. -> AbstractPlatformTransactionManager.getTransaction(txDef)
//     [if new] -> doBegin() -> bindResource(ds, connHolder)
//     [if existing+REQUIRES_NEW] -> suspend() -> SuspendedResourcesHolder
// 6. -> callback.proceedWithInvocation() // YOUR method runs
// 7. -> commitTransactionAfterReturning(txInfo)
//     -> AbstractPlatformTransactionManager.commit(status)
//     -> doCommit() -> con.commit()
//     -> triggerAfterCommit() synchronizations
// 8. [on exception] completeTransactionAfterThrowing(txInfo, ex)
//     -> txAttr.rollbackOn(ex) checks RollbackRuleAttribute list
//     -> doRollback() OR doCommit()
// 9. -> cleanupTransactionInfo(txInfo) - restores previous txInfo on ThreadLocal
```